

Non-Functional Requirements & Latency Numbers

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles

Functional requirements describe *what* a system does ("users can place an order"). **Non-functional requirements (NFRs)** describe *how well* it does it: how fast, how available, how many concurrent users, how consistent. Almost every HLD interview is graded far more on how you reason about NFRs than on the functional feature list — anyone can list features; understanding their performance and reliability implications is the actual skill being tested.

SLI, SLO, SLA — the reliability vocabulary

- **SLI (Service Level Indicator)**: the raw, measured metric — e.g., "P99 latency of the /checkout endpoint" or "percentage of successful requests."
- **SLO (Service Level Objective)**: your internal target for that metric — e.g., "P99 latency under 300ms, 99.9% availability over 30 days."
- **SLA (Service Level Agreement)**: the external, often contractual promise to customers, usually set slightly looser than your internal SLO as a safety margin — e.g., "we guarantee 99.9% uptime or you get a service credit."

Backend engineer analogy

If you've ever set up an uptime monitor or New Relic/Datadog dashboard tracking p95 response time on an API, you were tracking an SLI. The alert threshold you configured ("page me if p95 > 500ms") was an informal SLO. A formal SLA is the same idea, written into a contract with real consequences for missing it.

Understanding percentiles (P50, P95, P99)

Averages lie. If 99 requests take 50ms and 1 request takes 5000ms, the average looks fine (~99.5ms) but that one user had a terrible experience. **Percentiles** fix this: P50 (median) is the typical experience; P99 means 99% of requests were faster than this value — it captures your worst common case, which is exactly what angry users and SLAs care about. Interviewers frequently ask you to reason in P99 terms, not averages.

Latency numbers every engineer should know (approximate)

Operation	Approximate latency
L1 cache reference	~1 nanosecond
Main memory (RAM) reference	~100 nanoseconds
Redis / in-memory cache read (network)	~0.5–1 millisecond

Operation	Approximate latency
SSD random read	~100 microseconds
Read 1MB sequentially from SSD	~1 millisecond
Relational DB query (simple, indexed)	~1–10 milliseconds
Same-datacenter network round trip	~0.5–1 millisecond
Cross-region network round trip	~100–150 milliseconds

You don't need to memorize exact numbers — you need the *relative* intuition: memory is much faster than SSD, which is much faster than a database query, which is much faster than a cross-region network hop. This intuition is what lets you justify design decisions like "add a cache here" or "don't make a synchronous cross-region call in the critical path."

One more useful concept tied to SLOs: an **error budget**. If your SLO is 99.9% availability over 30 days, that means you're implicitly allowed about 43 minutes of downtime that month without breaching your objective — that allowance is your error budget. Teams use it deliberately: if the budget is nearly exhausted, they slow down risky releases and prioritize stability; if there's plenty of budget left, they can ship more aggressively. It reframes reliability from a vague goal into a concrete, spendable resource, which is a genuinely useful mental model to reference in interviews about production trade-offs.

2. Real-World Example / Case Study

Payment gateways like Razorpay publish explicit uptime SLAs to merchants (e.g., 99.9%+) because downtime directly costs their merchants real revenue — every minute the gateway is down, businesses can't collect payments. Internally, their engineering teams track much stricter SLOs (say, 99.95%) so that normal operational hiccups don't breach the customer-facing SLA — exactly the safety-margin relationship described above.

Careem's ride-matching flow likely has an internal SLO around P99 latency for "time to find a nearby driver" — because a P50 (median) view could look perfectly healthy while a meaningful chunk of users in low-driver-density areas experience painfully slow matches, directly hurting retention in exactly the markets where supply is already thin.

A second example: Noon's product search likely tracks a P95 or P99 latency SLO specifically (not just an average), because a slow search response during a peak sale event directly costs conversions — and since sale traffic is exactly when the system is most stressed and most likely to produce outliers, tracking only the average would systematically hide the exact failure mode leadership cares most about catching.

3. Diagram

Response times for 100 requests to /checkout (sorted):

```
Fastest -----> Slowest
[ 50 requests: ~40-60ms ] <- P50 sits here (median)
[ 44 requests: ~60-200ms ]
[ 5 requests: ~200-400ms ] <- P95 sits here
[ 1 request: ~4800ms ] <- P99 sits here (worst common case)
```

```
AVERAGE of all 100  =~ 91ms   <- looks fine, HIDES the outlier
P99                  = 4800ms <- reveals the real worst-case pain
```

4. Trade-offs Table

Metric choice	What it shows	What it hides
Average (mean)	General central tendency	Outliers and worst-case user pain
P50 (median)	The typical user experience	Everything the slower half experiences
P95 / P99	Near-worst-case experience, closer to real user pain	Requires more data/monitoring maturity to track well
Uptime % (e.g., 99.9)	Simple, contract-friendly reliability metric	Doesn't capture degraded-but-not-down performance

5. Common Interview Questions

Q1: "What's the difference between SLI, SLO, and SLA?"

Model approach: SLI is the raw measurement, SLO is your internal target for that measurement, SLA is the external contractual promise (usually looser than the SLO, as a buffer). Give one concrete example threading all three together for the same metric (e.g., latency).

Q2: "Why would you design around P99 latency instead of average latency?"

Model approach: Explain that averages can look healthy while a meaningful fraction of real users have a bad experience, and that at scale (millions of requests), even a "small" P99 tail represents a large absolute number of frustrated users — which is why production systems set alerts and SLOs on percentiles, not averages.

Q3: "What is an error budget, and how would you use it to make a release decision?"

Model approach: Define it as the allowed amount of unreliability implied by your SLO (e.g., 99.9% availability implies ~43 minutes of downtime allowed per month), then explain that a team tracks how much of that budget has been consumed — if it's nearly gone, they pause risky changes and focus on stability; if there's budget to spare, they can move faster. This ties reliability directly to a concrete, trackable number rather than a vague feeling.

6. Practice Exercises

- 1 Given response times [20ms, 22ms, 25ms, 30ms, 4000ms] for 5 requests, calculate the average and identify roughly which one represents the P99-ish outlier, and explain the gap.

- 2 Write a one-sentence SLO for a checkout API's availability and a separate one for its P99 latency.
- 3 Explain, using the latency table, why adding a Redis cache in front of a slow database query is likely to help far more than optimizing the query's SQL slightly.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"SLO & SLI Explained: Service Level Objectives & Indicators for Beginners"	Beginner-friendly breakdown of the SLI/SLO relationship.
English	"SLO vs SLI vs SLA vs Error Budget Google SRE in Plain English"	Ties the vocabulary to how Google's SRE practice actually uses it.
Hindi	"What is SLA in Hindi SLA kya hota hai Priority Severity"	Hindi explanation of SLA fundamentals and related support concepts.
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp	Covers NFRs and reliability concepts within a broader Hindi roadmap.

8. Key Takeaways

- Non-functional requirements (speed, availability, scale) are what HLD interviews mostly grade you on.
- SLI = measured metric, SLO = internal target, SLA = external contractual promise (usually looser than the SLO).
- Percentiles (P50/P95/P99) reveal real user pain that averages hide.
- P99 represents your near-worst-case experience and is what production systems actually alert on.
- Know the relative order of latency: memory < SSD < DB query < same-DC network < cross-region network.
- Design decisions (add a cache, avoid synchronous cross-region calls) are justified by this latency intuition.